FC7xxx WDG Integration Manual

Rev.0.4

Revision History

Revision	Date	Changes
0.1	2023/07/14	Initial release for MCAL V0.1.0
0.3	2023/10/20	Updated for MCAL V0.3.0
0.4	2023/12/15	Updated for MCAL V0.4.0 - Optimization plugins

Table of Contents

Chapter 1 Introduction

1.1 Introduction

This integration manual describes the integration requirements for the WDG module.

Chapter 2 Building

2.1 Dependencies on Other Modules

Module configuration dependency

- Mcu: This module provides the clock reference point for WDG module
- Det: This module is necessary for enabling Development error detection.
- Dem: This module provides the interface to set the related diagnostic event.
- Common: This module is the basic module which used to choose the chip.
- Gpt: This module provides periodic dog feeding function for WDG.
- EcuC: This module provides configuration for mapping WDG drivers to ECUC partitions (when multi-core functionality is enabled)

Module build dependency

- Common: This module provides common type for other modules.
- Rte: This module provides APIs to protect/unprotect some parts of code from interrupts (Exclusive Areas).
- Det: This module is necessary for enabling Development error detection.
- Dem: This module provides the interface to set the related diagnostic event.

Module initialization dependency

- Mcu: To use the WDG module, the user needs to initialize MCU module first.

2.2 Files Required for Compile

WDG module files:

- _MCAL/Src/Wdg/Src/Wdg_Instance0.c
- _MCAL/Src/Wdg/Src/Wdg_Instance1.c
- _MCAL/Src/Wdg/Src/Wdg_Instance2.c
- _MCAL/Src/Wdg/Src/Wdg_HLD.c
- _MCAL/Src/Wdg/Src/Wdg_HWA.c
- _MCAL/Src/Wdg/Src/Wdg_LLD.c
- _MCAL/Src/Wdg/Src/Wdg_Irq.c
- _MCAL/Src/Wdg/include/Wdg_HLD.h
- _MCAL/Src/Wdg/include/Wdg_HWA.h
- _MCAL/Src/Wdg/include/Wdg_Instance0.h
- _MCAL/Src/Wdg/include/Wdg_Instance0_Cbk.h
- _MCAL/Src/Wdg/include/Wdg_Instance1.h
- _MCAL/Src/Wdg/include/Wdg_Instance1_Cbk.h
- _MCAL/Src/Wdg/include/Wdg_Instance2.h
- _MCAL/Src/Wdg/include/Wdg_Instance2_Cbk.h
- _MCAL/Src/Wdg/include/Wdg_Irq.h
- _MCAL/Src/Wdg/include/Wdg_LLD.h
- _MCAL/Src/Wdg/include/Wdg_LLD_Types.h
- _MCAL/Src/Wdg/include/Wdg_MemMap.h

- _MCAL/Src/Wdg/include/Wdg_Reg.h
- _MCAL/Src/Wdg/include/Wdg_Version.h

- _MCAL_generate/src/Wdg_cfg.c
- _MCAL_generate/src/Wdg_Instance0_PBcfg.c
- _MCAL_generate/src/Wdg_Instance1_PBcfg.c
- _MCAL_generate/src/Wdg_Instance2_PBcfg.c
- _MCAL_generate/include/Wdg_Cfg.h

Common module files:

- _MCAL/Src/Common/include/Aontimer_Reg.h
- _MCAL/Src/Common/include/arm_cortex_asm.h
- _MCAL/Src/Common/include/Common_MemMap.h
- _MCAL/Src/Common/include/Compiler_Cfg.h
- _MCAL/Src/Common/include/CompilerDefinition.h
- _MCAL/Src/Common/include/Cpm_Reg.h
- _MCAL/Src/Common/include/Eth_GeneralTypes.h
- _MCAL/Src/Common/include/Fcpit_Reg.h
- _MCAL/Src/Common/include/Ftu_Common.h
- _MCAL/Src/Common/include/Ftu_Reg.h
- _MCAL/Src/Common/include/Ftu_RegOps.h
- _MCAL/Src/Common/include/Gpio_Reg.h
- _MCAL/Src/Common/include/IRQRouter.h
- _MCAL/Src/Common/include/Mb_Reg.h
- _MCAL/Src/Common/include/Mb_RegOps.h
- _MCAL/Src/Common/include/Mcal.h
- _MCAL/Src/Common/include/Platform_Types.h
- _MCAL/Src/Common/include/Port_Reg.h
- _MCAL/Src/Common/include/Scm_Reg.h
- _MCAL/Src/Common/include/Scm_RegOps.h
- _MCAL/Src/Common/include/SpinLock.h
- _MCAL/Src/Common/include/StdRegMacros.h
- _MCAL/Src/Common/src/Aontimer_Common.c
- _MCAL/Src/Common/src/Ftu_Common.c
- _MCAL/Src/Common/src/IRQRouter.c
- _MCAL/Src/Common/src/Port_Common.c
- _MCAL/Src/Common/src/SpinLock.c

Det module files:

- Det.h

Dem module files:

- Dem.h

Rte module files:

- SchM_Wdg.h

2.3 Add Plug-ins

WDG module plug-ins are developed for EB tresos Studio, so, to use WDG plug-ins on the EB tresos Studio, the user needs to add the WDG module to the EB plug-ins folder first.

1) Copy the WDG module(_MCAL/EB_Plugins/eclipse/plugins/Wdg) folder to EB tresos plug-ins (EB/tresos/plugins/) folder.
2) Set the WDG module output location folder for the generated source file and header files.
3) Use EB tresos to configure the WDG module and generate source and header files.

Chapter 3 Memory

3.1 Sections in Memory Map

Section Name	Section Type	Description
WDG_START_SEC_CONFIG_DATA_8 WDG_STOP_SEC_CONFIG_DATA_8 WDG_START_SEC_CONFIG_DATA_16 WDG_STOP_SEC_CONFIG_DATA_16 WDG_START_SEC_CONFIG_DATA_32 WDG_STOP_SEC_CONFIG_DATA_32	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code (data).
WDG_START_SEC_CONFIG_DATA_UNSPECIFIED WDG_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Start and stop of Memory Section for Config Data.
WDG_START_SEC_CONST_BOOLEAN WDG_STOP_SEC_CONST_BOOLEAN WDG_START_SEC_CONST_8 WDG_STOP_SEC_CONST_8 WDG_START_SEC_CONST_16 WDG_STOP_SEC_CONST_16 WDG_START_SEC_CONST_32 WDG_STOP_SEC_CONST_32 WDG_START_SEC_CONST_ UNSPECIFIED WDG_STOP_SEC_CONST_ UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit or boolean. These variables are read only (rodata).
WDG_START_SEC_CODE WDG_STOP_SEC_CODE WDG_START_SEC_ RAMCODE WDG_STOP_SEC_ RAMCODE WDG_START_SEC_CODE_AC WDG_STOP_SEC_CODE_AC	Code	Start and stop of memory Section for Code (text).
WDG_START_SEC_VAR_NO_INIT_BOOLEAN WDG_STOP_SEC_VAR_NO_INIT_BOOLEAN WDG_START_SEC_VAR_NO_INIT_8 WDG_STOP_SEC_VAR_NO_INIT_8 WDG_START_SEC_VAR_NO_INIT_16 WDG_STOP_SEC_VAR_NO_INIT_16 WDG_START_SEC_VAR_NO_INIT_32 WDG_STOP_SEC_VAR_NO_INIT_32 WDG_START_SEC_VAR_NO_INIT_ UNSPECIFIED WDG_STOP_SEC_VAR_NO_INIT_ UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are never cleared and never initialized by startup code (bss).
WDG_START_SEC_VAR_ INIT_BOOLEAN WDG_STOP_SEC_VAR_ INIT_BOOLEAN WDG_START_SEC_VAR_ INIT_8 WDG_STOP_SEC_VAR_ INIT_8	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by

Section Name	Section Type	Description
WDG_START_SEC_VAR _INIT_16 WDG_STOP_SEC_VAR _INIT_16 WDG_START_SEC_VAR _INIT_32 WDG_STOP_SEC_VAR _INIT_32 WDG_START_SEC_VAR _INIT_ UNSPECIFIED WDG_STOP_SEC_VAR _INIT_ UNSPECIFIED		startup code (data).

Chapter 4 Exclusive Area

WDG module using the services of Schedule Manger (SchM) for entering and exiting critical regions.

The following critical regions are used in the WDG driver:

- Wdg_HLD.c:
  - Wdg_HLD_ChannelTrigger: exclusive area 0
  - Wdg_HLD_Init: exclusive area 1
  - Wdg_HLD_SetMode: exclusive area 2
  - Wdg_HLD_SetTriggerCondition: exclusive area 3

Chapter 5 Interrupt Service Routine (ISR)

Instance	Interrupt Name	IRQ Number (NVIC Interrupt ID)
Watchdog 0	WDOG0_IRQHandler	37
Watchdog 1	WDOG1_IRQHandler	38
Watchdog 2	WDOG2_IRQHandler	39

Chapter 6 Error Report

6.1 Det

Function Name	**Error Type**
Wdg_Instance0_Init	WDG_E_DRIVER_STATE; WDG_E_INIT_FAILED; WDG_E_PARAM_MODE; WDG_E_PARAM_CONFIG; WDG_E_PARAM_TIMEOUT
Wdg_Instance0_SetMode	WDG_E_DRIVER_STATE; WDG_E_PARAM_MODE; WDG_E_PARAM_TIMEOUT
Wdg_Instance0_SetTriggerCondition	WDG_E_DRIVER_STATE; WDG_E_PARAM_TIMEOUT
Wdg_Instance0_GetVersionInfo	WDG_E_PARAM_POINTER;
Wdg_Instance1_Init	WDG_E_DRIVER_STATE; WDG_E_INIT_FAILED; WDG_E_PARAM_MODE; WDG_E_PARAM_CONFIG; WDG_E_PARAM_TIMEOUT
Wdg_Instance1_SetMode	WDG_E_DRIVER_STATE; WDG_E_PARAM_MODE; WDG_E_PARAM_TIMEOUT
Wdg_Instance1_SetTriggerCondition	WDG_E_DRIVER_STATE; WDG_E_PARAM_TIMEOUT
Wdg_Instance1_GetVersionInfo	WDG_E_PARAM_POINTER;
Wdg_Instance2_Init	WDG_E_DRIVER_STATE; WDG_E_INIT_FAILED; WDG_E_PARAM_MODE; WDG_E_PARAM_CONFIG; WDG_E_PARAM_TIMEOUT
Wdg_Instance2_SetMode	WDG_E_DRIVER_STATE; WDG_E_PARAM_MODE; WDG_E_PARAM_TIMEOUT
Wdg_Instance2_SetTriggerCondition	WDG_E_DRIVER_STATE; WDG_E_PARAM_TIMEOUT
Wdg_Instance2_GetVersionInfo	WDG_E_PARAM_POINTER;

6.2 Dem

Dem Event Name	Description
WDG_E_DISABLE_REJECTED	Shall be issued when the error "Initialization or mode switch failed because it would disable the watchdog" has occurred.
WDG_E_MODE_FAILED	Shall be issued when the error "Setting a watchdog mode failed (during initialization or mode switch)" has occurred.

Chapter 7 Function Calls

7.1 Function Calls during Startup

The API needs to be called is Wdg_Instance0_Init, Wdg_Instance1_Init, Wdg_Instance2_Init;

7.2 Function Calls during Shutdown

None

7.3 Function Calls during Wake-up

None

7.4 Function Calls during Runtime

None

Chapter 8 Other Requirements

8.1 Notification, Callback, Callout

- Notification

  If the user enables the watchdog interrupt and the notification value is not "NULL_PTR" or "NULL", an extern declaration will generate in Wdg_cfg.c. User can implement the notification in any file.

- Callback

  None

- Callout

  None

8.2 Macros

Please have a look in various definitions available in Common module's include file Mcal.h for details.

- If OS is not used:

  AUTOSAR_OS_NOT_USED need defined.

  In this case, If USE_SW_VECTOR_MODE is not defined, the user needs to map the interrupt vector table function to ISR function, for example：

  ```
  extern ISR(Wdg_Wdog0_Isr);

  void WDOG0_IRQHandler (void)

  {

  Wdg_Wdog0_Isr();

  }
  ```

- If OS is used:

  Do not define AUTOSAR_OS_NOT_USED.

Chapter 9 Integration Steps

1) Configure the WDG module and generate configuration files (please refer to Building chapter for details).
2) Configure appropriate memory sections in linker file or other (please refer to Memory chapter for details).
3) Map interrupt notification to their vector locations (please refer to chapter ISR for details).
4) Build the WDG module with dependent modules.